Function Study 1: apply(), map(), pipe()

Nov 4, 2026 · @tanvik

Overview

You've worked on `assign()`, number 1 on your list. Today's three are the next ones: `apply()` (2), `map()` (3, which also replaces the deprecated `applymap()`, number 4) and `pipe()` (5). Each one hands a function to pandas. What differs is **what that function receives**:

Function	Your function receives	Typical use
`s.map(...)`	One value at a time (or a lookup dict/Series)	Codes to names, lookups from another table
`s.apply(f)` / `df.apply(f)`	One value, one whole column, or one row	Custom logic with no built-in
`groupby(...).apply(f)`	One whole group	Logic that needs every row of a client
`df.pipe(f)`	The whole DataFrame	Chaining your own functions
`df.assign(new=f)`	The whole DataFrame	New columns inside a chain

Setup for every example and exercise

The examples run on the mastery workbook's practice data. Put this notebook in the same folder as `practice_data.py`:

```
import numpy as np
import pandas as pd
from practice_data import load

D = load()
led, calls, team = D['ledger'], D['calls'], D['team']
```

Progress through your list

Status	Functions
Done	1 `assign()`
Today	2 `apply()`, 3 `map()`, 4 `applymap()` (folded into `map`), 5 `pipe()`
Next session	6 `eval()`, 7 `replace()`
After that	8 `where()`, 9 `mask()`, then 10 `clip()`, 11 `query()`

Each later session will cover two functions, combined with the ones before them.

2. apply(): run your function on a column, a row or a group

`apply` calls your function and collects the results. What the function receives depends on where you call it, and that decides whether it's fast or slow.

Call	Function receives	Speed
`df[cols].apply(f)`	Each **column** (a Series)	Fast: runs once per column
`df.groupby(key)[col].apply(f)`	Each **group's** values	Fine: runs once per client
`s.apply(f)`	Each **value**	Slow: once per row
`df.apply(f, axis=1)`	Each **row**	Slowest: once per row, as a Series

On columns: one function across several columns

```
messy = led[['Debit', 'Credit', 'Balance']].astype(str)          # numbers
stored as text
clean = messy.apply(pd.to_numeric, errors='coerce')              # nested:
apply + to_numeric
clean.dtypes                                                       # all
float64

led[['Debit', 'Credit']].apply(lambda s: s.eq(0).sum())          # zeros per
column: Debit 2767, Credit 2690
```

- **`apply(pd.to_numeric, errors='coerce')`:** extra keyword arguments after the function are passed on to it. Every column is converted in one line.
- **`lambda s: s.eq(0).sum()`:** `s` is a whole column, so the work inside stays vectorised.

On groups: logic that needs all of a client's rows

```
longest_gap = (led[led['LedgerType'].eq('BILL')]
                 .assign(day=lambda d: d['CreatedAt'].dt.normalize())
                 .groupby('ClientID')['day']
                 .apply(lambda s:
s.drop_duplicates().sort_values().diff().max().days))
# C100: 6 days; the longest break of any client is 29 days (C113)
```

This nests `assign` + `groupby` + `apply` + `drop_duplicates` + `sort_values` + `diff` + `max`. Each client's trading days arrive as one Series, and the lambda finds the biggest gap between consecutive days. No built-in does that in one name, so `apply` is the right tool.

On rows: works, but look for the vectorised version

```
head = led.head(3)
slow = head.apply(lambda r: f"{r['ClientID']} | {r['LedgerType']} |
{r['CreatedAt']:%Y-%m-%d}", axis=1)
fast = head['ClientID'] + ' | ' + head['LedgerType'] + ' | ' +
head['CreatedAt'].dt.strftime('%Y-%m-%d')
(slow == fast).all()                                               # True:
same text, no row loop
```

`axis=1` hands each row over as a Series, so `r['ClientID']` works. Use it when the logic really needs several columns and has no column form. Otherwise build the result from whole columns, which was 900 times faster in the earlier timing.

Pitfalls

- **`axis`:** the default `axis=0` means "each column". Forgetting `axis=1` for row logic gives a `KeyError` on a column name.
- **`s.apply(lambda x: x * 2)`** works, but `s * 2` is the same, 180 times faster.
- **Return one thing per call.** A lambda that returns a Series on `axis=1` creates several columns, which is useful but surprising when unintended.

3. map(): translate each value

`map` replaces each value in a column with something else: a label from a dictionary, a value looked up in another table, or the result of a function. The column keeps its length and order.

What you pass	What happens	Values not found
A dict	Look each value up as a key	Become NaN
A Series	Look each value up in its index	Become NaN
A function	Call it on each value	Whatever the function returns

With a dict: codes to categories

```
CATEGORY = {'BILL': 'Trading', 'DEPOSIT': 'Money in', 'WITHDRAW': 'Money
out'}

led = led.assign(Category=lambda d:
d['LedgerType'].map(CATEGORY).fillna('Other'))   # nested: assign + map +
fillna
led['Category'].value_counts()            # Trading 5292, Money in 108, Money
out 56
```

Any ledger type missing from the dict becomes NaN, and `.fillna('Other')` turns it into a label. Without that, new types (such as `WITHDRAW CANCELLED`) disappear quietly from later groupings.

With a Series: a lookup from another table

```
tl_lookup = team.set_index('CRM ID')['TL']                  # index = key,
values = answer
calls = calls.assign(TL=lambda d: d['CRM ID'].map(tl_lookup))
calls.groupby('TL').size()                                  # TL A 337, TL B
211, TL C 136
```

This is a one-column left merge: no new rows, no `_x` / `_y` suffixes, and the result lines up with `calls`. **The lookup's index must be unique.** If one CRM ID appears twice in `team`, `map` raises `InvalidIndexError: Reindexing only valid with uniquely valued Index objects`. Fix the team list with `drop_duplicates('CRM ID')` first.

With a function

```
calls['CRM ID'].map(lambda e: e.split('@')[0])              #
'amrut@nexora.live' -> 'amrut'
pd.Series(['C100', None]).map(str.lower, na_action='ignore')  # ['c100',
NaN]: blanks skipped
```

`na_action='ignore'` skips blanks instead of passing them to your function, so `None.lower()` can't crash it.

DataFrame.map: every cell (the old applymap)

```
summary = led.groupby('ClientID')[['Debit', 'Credit']].sum()
summary.map(round)                                           # every cell
rounded: C100 Debit 2679, Credit 6607
```

`applymap()` is deprecated; `DataFrame.map()` does the same job. For plain rounding, `summary.round()` is faster. Use `DataFrame.map` for custom per-cell logic, such as stripping spaces only where a cell is text: `df.map(lambda v: v.strip() if isinstance(v, str) else v)`.

Pitfalls

- **Silent NaN:** a typo in a dict key leaves values blank without an error. Check `.isna().sum()` after mapping.
- **Spaces and case:** `'BILL '` doesn't match `'BILL'`. Clean first: `s.str.strip().str.upper().map(...)`.

4. pipe(): chain your own functions

`df.pipe(f, a, b=1)` is the same as `f(df, a, b=1)`. The point is the order you read it in: steps written top to bottom, in the order they happen, instead of nested inside-out. It's how the functions you're learning to write become steps in a chain.

```
def in_range(df, start, end, col='CreatedAt'):
    """Rows with start <= col < end."""
    return df[(df[col] >= start) & (df[col] < end)]

def only_type(df, ledger_type):
    """Rows of one ledger type."""
    return df[df['LedgerType'].eq(ledger_type)]

def trading_days(df):
    """Distinct trading dates per client."""
    return (df.assign(day=df['CreatedAt'].dt.normalize())
              .groupby('ClientID', as_index=False)['day'].nunique()
              .rename(columns={'day': 'Trading Days'}))

oct_days = (led
            .pipe(in_range, '2026-10-01', '2026-11-01')    # October rows
only
            .pipe(only_type, 'BILL')                       # 2,256 October
BILL rows
            .pipe(trading_days))                           # 40 clients; C100
traded on 8 days
```

Without pipe, the same thing reads inside-out

```
trading_days(only_type(in_range(led, '2026-10-01', '2026-11-01'), 'BILL'))
```

You have to read it from the innermost bracket outwards, and an argument sits far from its function.

Rules for functions you pipe

- **The first parameter receives the DataFrame.** Everything else is passed after it: `.pipe(only_type, 'BILL')`.
- **Return a DataFrame**, so the next `.pipe` or `.assign` has something to work on.
- **Don't change the input.** Build and return a new table; never use `inplace=True` inside.

When the DataFrame isn't the first parameter

```
def save_csv(path, data):
    data.to_csv(path, index=False)
    return data

oct_days.pipe((save_csv, 'data'), r'C:\...\oct_days.csv')     # (function,
name of the DataFrame parameter)
```

The tuple form tells `pipe` which parameter should receive the DataFrame.

Debugging with pipe

```
def peek(df, label=''):
    print(f"{label}: {len(df):,} rows")
    return df

led.pipe(in_range, '2026-10-01', '2026-11-01').pipe(peek,
'October').pipe(only_type, 'BILL').pipe(peek, 'BILL')
```

5. All four together

Each function does one job in a chain: `pipe` runs your own steps, `assign` adds columns, `map` translates values, and `apply` handles the per-group logic that has no built-in.

Money in, money out and trading result per client, for October

```
CATEGORY = {'BILL': 'Trading', 'DEPOSIT': 'Money in', 'WITHDRAW': 'Money
out'}

oct_flow = (led
    .pipe(in_range, '2026-10-01', '2026-11-01')
# your function
    .assign(Category=lambda d: d['LedgerType'].map(CATEGORY).fillna('Other'),
# map inside assign
            Amount=lambda d: d['Credit'] - d['Debit'])
# + in, - out
    .pivot_table(index='ClientID', columns='Category', values='Amount',
                 aggfunc='sum', fill_value=0)
    .reset_index())
# C100: Money in 2,988.00, Money out 0.00, Trading -408.41
```

Each TL's agent with the most connected calls

```
best = (calls
    .assign(TL=lambda d: d['CRM ID'].map(team.set_index('CRM ID')['TL']),
# lookup with map
            connected=lambda d: d['Call Status'].eq('connected'))
    .groupby(['TL', 'CRM ID'], as_index=False)['connected'].sum()
    .sort_values(['TL', 'connected'], ascending=[True, False])
    .drop_duplicates('TL'))
# top agent per TL
# TL A: amrut (51), TL B: neha (46), TL C: uno8943 (62)
```

The same "top per group" can be written with `apply`: `.groupby(level='TL').apply(lambda s: s.idxmax())`. The sort and `drop_duplicates` version is easier to read and keeps the count beside the name.

Choosing between them

You need	Reach for
A new column, inside a chain	`assign`
Replace values using a dict or another table	`map`

You need	Reach for
A custom step you'll reuse	`def` + `pipe`
Custom logic on a whole group	`groupby().apply`
The same function on several columns	`df[cols].apply(f)`
Logic across several columns of one row	Column arithmetic first; `apply(axis=1)` if nothing else fits

6. Exercises

Write each one without looking at the examples, then compare with the checkpoint. The checkpoints come from the practice data, so a match means your code is right.

apply()

- [ ] **A1.** Make `messy = led[['Debit', 'Credit', 'Balance']].astype(str)`, then convert all three columns back to numbers in **one** line with `apply` and `pd.to_numeric`. *Checkpoint: every dtype is `float64`.*
- [ ] **A2.** Count the zero values in `Debit` and in `Credit` with one `apply` on both columns. *Checkpoint: Debit 2767, Credit 2690.*
- [ ] **A3.** For each client, find the longest break in days between two consecutive BILL **days**. *Checkpoint: C100 is 6; the largest is C113 with 29.*
- [ ] **A4.** Build the text `'C127 | BILL | 2026-08-03'` for the first 3 ledger rows twice: with `apply(axis=1)`, and with column `+` only. Prove they're equal. *Checkpoint: `(a == b).all()` is `True`.*

map()

- [ ] **M1.** Add `Category` with `map` and the `CATEGORY` dict, inside `assign`, with blanks as `'Other'`. *Checkpoint: Trading 5292, Money in 108, Money out 56.*
- [ ] **M2.** Add the TL to every call by mapping `CRM ID` through the team list, then count calls per TL. *Checkpoint: TL A 337, TL B 211, TL C 136.*
- [ ] **M3.** Turn every CRM ID into a short name (`'amrut@nexora.live'` → `'amrut'`) with `map` and a function. *Checkpoint: 7 distinct short names, the first is `'abhishek'`.*
- [ ] **M4.** Sum Debit and Credit per client, then round **every** cell to a whole number with `DataFrame.map`. *Checkpoint: C100 Debit 2679, Credit 6607.*

- [ ] **M5.** Break the lookup on purpose: add one team row twice and map again. Read the error, then fix it with `drop_duplicates`. *Checkpoint:* `InvalidIndexError`*, then it works.*

pipe()

- [ ] **P1.** Write `in_range(df, start, end)` and `only_type(df, ledger_type)`, then chain them with `pipe` for October BILL rows. *Checkpoint: 2,256 rows.*
- [ ] **P2.** Write `trading_days(df)` and add it to the P1 chain. *Checkpoint: 40 clients; C100 has 8.*
- [ ] **P3.** Write `peek(df, label)` and put it between every step of P2, printing the row count each time.

Combinations

- [ ] **C1.** In one chain, using `pipe`, `assign`, `map` and `pivot_table`: October money in, money out and trading result per client, with `Amount = Credit - Debit`. *Checkpoint: C100 Money in 2,988.00, Money out 0.00, Trading -408.41.*
- [ ] **C2.** In one chain: map each call's TL, flag connected calls, and find each TL's agent with the most connected calls. *Checkpoint: TL A amrut (51), TL B neha (46), TL C uno8943 (62).*
- [ ] **C3 (stretch).** Turn C1 into a function `money_flow(df, start, end)` that uses `pipe` inside, then call it for September too.

7. Answers

Try each exercise first. All of these were run on the practice data and match the checkpoints. After reading one, close it and write it again from memory.

apply()

```
# A1
messy = led[['Debit', 'Credit', 'Balance']].astype(str)
clean = messy.apply(pd.to_numeric, errors='coerce')

# A2
led[['Debit', 'Credit']].apply(lambda s: s.eq(0).sum())

# A3
gap = (led[led['LedgerType'].eq('BILL')]
         .assign(day=lambda d: d['CreatedAt'].dt.normalize())
         .groupby('ClientID')['day']
         .apply(lambda s:
s.drop_duplicates().sort_values().diff().max().days))
gap['C100'], gap.idxmax(), gap.max()

# A4
head = led.head(3)
a = head.apply(lambda r: f"{r['ClientID']} | {r['LedgerType']} |
{r['CreatedAt']:%Y-%m-%d}", axis=1)
b = head['ClientID'] + ' | ' + head['LedgerType'] + ' | ' +
head['CreatedAt'].dt.strftime('%Y-%m-%d')
(a == b).all()
```

map()

```
# M1
CATEGORY = {'BILL': 'Trading', 'DEPOSIT': 'Money in', 'WITHDRAW': 'Money
out'}
led = led.assign(Category=lambda d:
d['LedgerType'].map(CATEGORY).fillna('Other'))
led['Category'].value_counts()

# M2
calls = calls.assign(TL=lambda d: d['CRM ID'].map(team.set_index('CRM ID')
['TL']))
calls.groupby('TL').size()

# M3
calls['CRM ID'].map(lambda e: e.split('@')[0])
```

```
# M4
led.groupby('ClientID')[['Debit', 'Credit']].sum().map(round)

# M5
bad = pd.concat([team, team.head(1)])
# calls['CRM ID'].map(bad.set_index('CRM ID')['TL'])        ->
InvalidIndexError
fixed = bad.drop_duplicates('CRM ID')
calls['CRM ID'].map(fixed.set_index('CRM ID')['TL'])
```

pipe()

```
# P1
def in_range(df, start, end, col='CreatedAt'):
    return df[(df[col] >= start) & (df[col] < end)]

def only_type(df, ledger_type):
    return df[df['LedgerType'].eq(ledger_type)]

oct_bills = led.pipe(in_range, '2026-10-01', '2026-11-01').pipe(only_type,
'BILL')

# P2
def trading_days(df):
    return (df.assign(day=df['CreatedAt'].dt.normalize())
              .groupby('ClientID', as_index=False)['day'].nunique()
              .rename(columns={'day': 'Trading Days'}))

oct_days = oct_bills.pipe(trading_days)

# P3
def peek(df, label=''):
    print(f"{label}: {len(df):,} rows")
    return df

(led.pipe(peek, 'start')
    .pipe(in_range, '2026-10-01', '2026-11-01').pipe(peek, 'October')    #
2,328 rows
    .pipe(only_type, 'BILL').pipe(peek, 'BILL')                           #
2,256 rows
    .pipe(trading_days).pipe(peek, 'clients'))                            #
40 rows
```

Combinations

```
# C1 and C3
def money_flow(df, start, end):
    out = (df.pipe(in_range, start, end)
             .assign(Category=lambda d:
d['LedgerType'].map(CATEGORY).fillna('Other'),
                     Amount=lambda d: d['Credit'] - d['Debit'])
             .pivot_table(index='ClientID', columns='Category',
values='Amount',
                          aggfunc='sum', fill_value=0)
             .reset_index())
    out.columns.name = None
    return out

oct_flow = money_flow(led, '2026-10-01', '2026-11-01')
sep_flow = money_flow(led, '2026-09-01', '2026-10-01')

# C2
best = (calls
    .assign(TL=lambda d: d['CRM ID'].map(team.set_index('CRM ID')['TL']),
            connected=lambda d: d['Call Status'].eq('connected'))
    .groupby(['TL', 'CRM ID'], as_index=False)['connected'].sum()
    .sort_values(['TL', 'connected'], ascending=[True, False])
    .drop_duplicates('TL'))
```

Next session, when you ask: 6 `eval()` and 7 `replace()`, combined with today's four.